

Foundations of Artificial Intelligence

M. Helmert
C. Büchner

University of Basel
Spring Term 2026

Exercise Sheet 6

Due: April 12, 2026

Please note the submission rules at the end of this exercise sheet. Non-adherence to these rules may lead to your submission not being marked.

Due to the easter break, you have two weeks to work on this exercise sheet and it covers 1.5 times the usual content (i.e., 6 slide sets and exercises worth 15 marks).

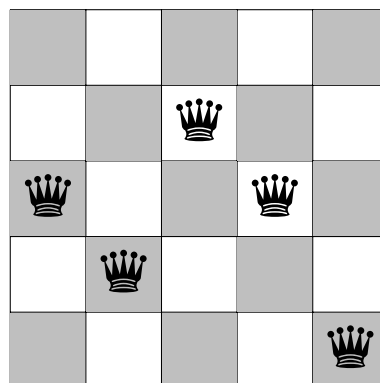
Exercise 6.1 (1+0.5+0.5 marks)

A *Hamiltonian path* of a directed graph $G = \langle V, E \rangle$ is a path through G that visits each vertex $v \in V$ exactly once. The *maximum Hamiltonian path* of a directed graph G with (positive) weighted edges is the Hamiltonian path with the maximal sum of edge weights.

- (a) Formalize the combinatorial optimization problem (COP) of finding a maximum Hamiltonian path in a fully connected graph, i.e., there is an edge between all pairs of vertices in the graph.
- (b) Is your formulation a pure search problem, a pure optimization problem, or a combined search and optimization problem?
- (c) Define a suitable neighborhood for hill climbing by providing a formal definition of a function mapping a candidate to a set of neighbors. A neighborhood function is suitable for hill climbing if it ensures that every candidate can be reached and if it bounds the amount of neighbors to a reasonable size. You do not need to show that the function is suitable.

Exercise 6.2 (1.5+1.5 marks)

The 5-queens problem is the COP on a 5×5 chess board that is analogous to the 8-queens problem that has been introduced in Chapter C1 in the lecture. In this exercise, we are interested in finding a solution for the 5-queens problem with different local search variants. Consider the following initial candidate:



- (a) Depict the candidates that are considered by hill climbing with the heuristic and the neighborhood that are introduced on slide 23 of Chapter C1 in the print version of the lecture slides. If necessary, break ties in favor of candidates where the leftmost possible queen is moved to a square that is as far up as possible. Annotate the candidates that are considered during search with the heuristic values of all neighbors.

- (b) Now consider a hill climbing variant where the next candidate that is considered is selected by first picking a file at random and then moving the (unique) queen in that file to a square in that file with the best heuristic value (the variant is introduced on slide 9 of Chapter C2 in the print version of the lecture slides). Assume that the random choices are resolved in such a way that the file selection picks the third file first, and in each subsequent step the file that is to the left of the file that was selected last (if the leftmost file was selected last, the rightmost file is selected). Provide the candidates that are considered if the heuristic is used that counts the number of queens that threaten the queen in the selected file. If necessary, break ties in favor of candidates where the queen that is moved is as far up as possible. Annotate the candidates that are considered during search with the heuristic values of all neighbors.

Note that the file chess-board.tex, which you can find on the ADAM website, contains the LaTeX source code for the 5×5 chess board depicted above.

Exercise 6.3 (2+0.5+0.5 marks)

Consider the propositional formula $\varphi = (a \vee b) \wedge (\neg b \vee c) \wedge (\neg a \vee \neg b)$.

- Formalize φ as a binary constraint network.
- Provide a consistent partial variable assignment.
- Provide an inconsistent partial variable assignment.

Exercise 6.4 (1.5+0.5+0.5+1.5 marks)

Consider the constraint network $\mathcal{C} = \langle V, \text{dom}, (R_{uv}) \rangle$ with $V = \{w, x, y, z\}$ and

$$\begin{array}{ll} \text{dom}(w) = \{1, 2, 3, 4\} & R_{wy} = \{\langle 3, 3 \rangle\} \\ \text{dom}(x) = \{1, 2, 3\} & R_{wz} = \{\langle 1, 4 \rangle, \langle 2, 4 \rangle, \langle 3, 2 \rangle, \langle 4, 4 \rangle\} \\ \text{dom}(y) = \{1, 3, 4\} & R_{xz} = \{\langle 1, 4 \rangle, \langle 2, 2 \rangle\} \\ \text{dom}(z) = \{2, 4\} & R_{yz} = \{\langle 3, 2 \rangle, \langle 3, 4 \rangle\} \end{array}$$

- Provide the search tree that is created by applying naive backtracking on $\mathcal{C}$ as introduced in chapter D3. Choose variables in alphabetical order and values in ascending order. Depict the search tree in the same style as the lecture slides, with nodes ordered from left to right according to the order in which they are explored by the algorithm.
- Consider the following static order strategy for the variables:
 - Select a variable according to the *minimum remaining values* variable order criterion.
 - If there is more than one such variable, break ties according to the *most constraining variable* variable order criterion.
 - If the choice is still not unique, break ties alphabetically.

Provide the resulting variable order.

- For each variable in the order of part (b), order its values according to the *minimum conflicts* criterion (number of conflicts with *unassigned* variables, i.e., variables later in the variable order). Break ties in favor of lower values as in part (a).
- Now provide the search tree that is created by applying naive backtracking using the variable order from (b) and the value order from part (c). Compare the result to the result of (a).

Exercise 6.5 (1+2 marks)

Consider the constraint network $\mathcal{C} = \langle V, \text{dom}, (R_{uv}) \rangle$ with

- $V = \{a, b, c\}$
- $\text{dom}(x) = \{0, 1, 2, 3\}$ for all $x \in V$
- $R_{a,b} = \{\langle 0, 0 \rangle, \langle 1, 1 \rangle, \langle 2, 2 \rangle, \langle 3, 3 \rangle\}$
 $R_{a,c} = \{\langle 0, 3 \rangle, \langle 1, 2 \rangle, \langle 2, 1 \rangle, \langle 3, 0 \rangle\}$
 $R_{b,c} = \{\langle 0, 1 \rangle, \langle 0, 2 \rangle, \langle 0, 3 \rangle, \langle 1, 2 \rangle, \langle 1, 3 \rangle, \langle 2, 3 \rangle\}$

- Consider the partial assignment $\alpha = \{b \mapsto 0\}$. Specify the domains of all variables after applying forward checking in α .
- Define the tightest network $\mathcal{C}'$ that is equivalent to $\mathcal{C}$. In other words, find the unique $\mathcal{C}'$ such that $\mathcal{C}' \equiv \mathcal{C}$ and there is no $\mathcal{C}''$ with $\mathcal{C}'' \subset \mathcal{C}'$ and $\mathcal{C}'' \equiv \mathcal{C}$. Justify your solution.

Submission rules:

- Submit one solution in groups of up to two students, stating the names of all group members in the submission. All group members are equally responsible for all submitted exercises.
- Create a single A4 PDF file for all non-programming exercises. Solutions in L^AT_EX are preferred, but clearly legible scans in A4 with reasonable contrast and file size are accepted as well.
- In case there are programming exercises, upload a ZIP file (ending .zip, .tar.gz or .tgz; *not* .rar or anything else) including your PDF and those code textfiles required by the exercise. Put your names in a comment on top of each code file. Make sure your code compiles and test it. Code that does not compile or which we cannot successfully execute will not be graded.
- Include a statement on scientific integrity at the end of your submission (see below).

Statement on scientific integrity:

- All submitted solutions must be the product of your own work.
- If your written solutions were produced using any help (e.g., from peers or a large language model), list in what way these people or tools contributed to the solutions.
- We do not mark submissions that lack a statement on scientific integrity.
- Examples:
 - “We solved all exercises on our own.”
 - “We used ChatGPT to explain exercise 7 which we did not understand.”
 - “We had no time to solve this exercise sheet and copied all solutions from our friends.”
(If this is the case, thanks for your honesty but this is plagiarism, results in a warning and no marks. It is preferable to submit no solution at all.)